

Lab 4.2: Robustness, Attacks and Honest Explanations

Note: this lab has no example code. Every step says what the code must do, and most steps add a hint. Read Sections 1 to 4 first, then do Parts A to F in order. You reuse the CICIDS data and the models from Lab 1 so nothing new has to be downloaded.

1. What is this lab about?

In Lab 4.1 you asked *why* a detector made a decision. That question assumed the detector was right. This lab asks a harder pair of questions: **does the detector still work when the data are imperfect**, and **does it still work when someone is trying to break it?**

You work on network flows this time, not URLs. The problem is: is this network flow an attack or normal traffic?

- **Part A: measure a baseline**, including the score you get for predicting “normal” every single time.
- **Part B: break the data by accident** : add measurement noise, and lose some features, as a real sensor would.
- **Part C: break the model on purpose** : an evasion attack, where the attacker changes only the things a real attacker can change.
- **Part D: compare** the features SHAP says are important with the features the attack actually used.
- **Part E: two defences**: an ensemble, and finding which groups of features carry the real information.
- **Part F: test the explanations themselves** : three checks that show whether SHAP and LIME are describing the model or just sounding convincing.

2. What you will be able to do afterwards

- Say how much of a model's score is real and how much is the class imbalance, and pick the right metric to report.
- Measure how fast a model degrades under noise and under missing features, and say which model is safest.
- Write an evasion attack that obeys the limits of the network, and explain why an unconstrained attack overstates the risk.
- Say whether a model leans on evidence an attacker can fake or evidence they cannot.
- Test whether an explanation is faithful, not just stable, and explain what correlated features do to SHAP.

3. The ideas, explained simply

The examples use made up numbers; your own numbers will be different. In this lab always report macro-F1, recall, PR-AUC and FAR.

3.1 The score that means nothing

The trivial baseline. Attacks are rare in network traffic. If 2 in every 100 flows are attacks, a model that answers “normal” to everything is 98% accurate and catches nothing. A score only means something next to that baseline.

example: 10,000 test flows, 200 attacks

Always normal: accuracy 0.980, recall 0.000

Your forest: accuracy 0.994, recall 0.780

The accuracy went up by 0.014. The recall went from nothing to 0.78. **Recall is where the real improvement is.**

ROC-AUC . ROC-AUC can look excellent on imbalanced data because it is dominated by the many normal flows.

3.2 Robustness: the score after the data get messy

A sensor does not hand you clean numbers. Timers drift, counters are sampled, and a collector under load drops fields. So we take the clean test set, damage it on purpose in two ways, and measure what is lost.

- **Noise.** Add a small random amount to each continuous feature. The amount is a percentage of that feature's standard deviation, so it is small for steady features and larger for variable ones.

- **Missing values.** Replace some values with the training median, which is what a model should assume when it is told nothing.

A model that loses very little is robust. A model that collapses when relying on precision it will never have in production.

3.3 Evasion: the attacker moves

Noise is accidental. An evasion attack is deliberate: the attacker changes the features of their own traffic until the model calls it normal. The label does not change, the flow is still an attack. Only the measurements change.

Greedy search. At each step, try every change you are allowed to make, and keep whichever one lowers the attack probability most. Repeat five times. It is simple, and close to what a real attacker does by trial and error.

3.4 What a network attacker can actually change

This is the part that makes the attack honest. The attacker sends the forward traffic, so they can hide packets and wait longer. They do not control the victim's replies, and they cannot finish the attack with fewer packets than the payload needs. Sort every feature into one of three groups:

- **Free.** Forward packet sizes, forward header and segment size, flow and forward inter-arrival times, idle and active timers, flow duration. The attacker can increase these. Padding a packet and waiting longer cost almost nothing.
- **Costly.** Total forward packets, total forward bytes, forward subflow counts, forward data packets. The attacker can increase these too, but each increase is more work and a louder attack.
- **Fixed.** Every backward-direction feature, destination port, protocol, initial backward window, and any flag count the responder sets. The victim's network stack produces these, so the attacker cannot touch them.

An attack that ignores these groups will make evasion look far easier than it is. You run the attack **both ways** in Part C so you can see the difference for yourself.

3.5 Stable is not the same as faithful

Stable. Run the explainer twice on almost the same input and get almost the same answer. LIME is random, so this is a real question for LIME.

Faithful. The explanation describes what the model actually did. An explanation can be perfectly stable and still wrong, because it is stable about the wrong thing.

The deletion test. If SHAP says a feature is important, replacing it with an ordinary value should push the score down. If it does not, either the ranking is wrong or something else is quietly covering for it. Compare the drop from the top-ranked features with the drop from the same number of random features: the gap between the two is the fidelity.

3.6 Twins: why SHAP splits the credit

On network flows, Fwd Pkt Len Mean, Max, Min and Std are close to four measurements of the same thing. When several features carry the same information, SHAP has to divide the credit between them, and how it divides is partly arbitrary. The total is still exactly right; the individual numbers can mislead.

Made-up example: two twins

SHAP says feature A pushes +0.30 and its twin B pushes +0.05.

Remove A alone: the score barely moves, because B still carries the signal.

Remove A and B together: the score falls a long way.

A looked unimportant when tested alone, and essential when tested with its twin.

So in Part F you run the deletion test twice: once on single features, and once on whole groups of correlated features. Part E does the same for feature importance.

4. Setup and dataset

Dataset: CICIDS2018 network flows — the same file you cleaned in Lab 1. Re-run your Lab 1 loading and cleaning so that `X_train`, `X_val`, `X_test` and the matching labels exist, with the label as `1 = attack` and `0 = benign`. Then train the three models on them: a decision tree, a logistic regression and a random forest of 300 trees. Keep `random_state=42`.

Libraries. The same as Lab 4.1: `shap` and `lime`, plus `scipy` for the rank correlation in Part D.

5. Part A: The baseline

Step A1: Check the data is healthy

Flow data is messier than the phishing table. Two divisions by zero and a few very large values are normal, and they will ruin later steps if you leave them in.

What to write

Print the shape of the training and test sets, and the share of attacks in the training labels.

Check that no infinite or missing values remain. `np.isfinite(X_train).all().all()` covers both at once.

Count how many features are continuous (more than two distinct values) and how many are 0/1 flags.

What you should see

Almost every feature continuous, and only the flag-count columns near-binary. This is the opposite of the phishing data, where many features were flags.

Flow `Byts/s` and `Flow Pkts/s` are the usual source of infinities. If any survive, fix them here before going on.

Step A2: Score the three models, and the model that does nothing

Take your `report()` function and add PR-AUC to it, from `average_precision_score`. Then score the tree, the logistic regression and the forest on the clean test set.

What to write

Add PR-AUC to `report()`, so it returns macro-F1, recall, ROC-AUC, PR-AUC and FAR.

Run it on all three models with the clean test set.

Work out the accuracy and the recall of a model that always answers benign. The accuracy is `1 - y_test.mean()` and the recall is zero.

What you should see

Scores near 0.99 for the forest. The always-benign model will also look good on accuracy alone.

Write both down. Every later number in this lab is compared against them.

Check yourself

How far above the always-benign accuracy is your forest? Is that margin as impressive as the raw accuracy made it look?

Compare ROC-AUC with PR-AUC for the same model. Which one moved more, and why?

6. Part B: Break the data by accident

Step B1: Add noise

Write a function that adds a small random amount to every continuous feature and leaves the 0/1 flags alone. The noise width for a column is a chosen level multiplied by that column's standard deviation.

What to write

A function taking the data, a reference set, a level and a seed, returning a noisy copy. Measure the standard deviation on **the training set**, never the test set, using the test set would be leakage.

Run all three models at levels 0.00, 0.05, 0.10, 0.20 and 0.50, and print macro-F1 and recall at each level.

Hint

Flow features are heavy-tailed: a few flows have an enormous Flow Duration. The same 5% level is therefore a much bigger absolute change on those rows. Expect the damage to be uneven.

Durations, counts and byte totals cannot be negative in reality. Decide whether to clip at zero, and write your reason in a comment.

What you should see

At 5% the forest should lose very little and the single tree the most.

Watch recall as well as macro-F1. On imbalanced data the F1 can hold up while recall quietly falls.

Check yourself

Why does a single tree fall apart faster than a forest of 300 trees?

Did clipping at zero change your results? Is a negative packet count a fair test of a model?

Step B2: Lose some features

Now simulate a collector that drops fields: replace a random tenth of each row's values with the training median.

What to write

A function taking the data, a fraction and a seed, replacing that fraction of each row with the training-set median.

Run it at 10% and compare all three models against the clean baseline.

Check yourself

Which does more damage, noise or missing values?

In a real collector, features do not go missing one at a time. A whole group fails together, such as every backward-direction field, or every idle timer. Would that be worse than what you simulated?

7. Part C: Break the model on purpose

Now use the three groups from Section 3.4. The attack is greedy: five steps, and at each step you keep whichever single permitted change lowers the attack probability most.

Step C1: Sort the features and set a budget

What to write

Assign every feature to Free, Costly or Fixed, and print how many landed in each group.

A function `can_change(feature, old_value, new_value)` returning True only when the feature is Free or Costly **and** the new value is at least as large as the old one.

A budget: one feature may rise by at most half of its training standard deviation in a single step, and at most five features may change in total.

Hint

Sort by name pattern, not by hand. Anything starting with Bwd is Fixed; anything containing IAT, Idle or Active is Free.

If you are unsure about a feature, call it Fixed. An attack that is weaker than reality is a safer mistake than one that lets the attacker do the impossible.

Step C2: Run the attack, twice

What to write

Pick the 50 test flows the forest is most confident are attacks.

Run the greedy search on each: five steps, trying every permitted increase, rejecting anything that breaks a constraint, stopping early if nothing helps.

Report how many of the 50 fell below probability 0.5, the average probability before and after, and the average number of features changed.

Then run the whole thing again with every constraint switched off, and put the two sets of results **side by side**.

Hint

Copy each row before you edit it, or you will damage the test set itself.

What you should see

The unconstrained attack should evade far more flows than the constrained one. That difference is the point of the step.

The constrained attack should still succeed on some flows. Those are the ones worth looking at.

If the constrained attack evades nothing at all, your groups are too strict. Move a few features from Fixed to Costly and say that you did.

Check yourself

The gap between the two runs is protection that comes from the network, not from your model. Which of the two numbers would you quote to a customer, and which to your own security team?

Step C3: See what the attack touched

What to write

Count how often the constrained attack changed each feature across all 50 flows, and print the ten most frequent, with the group each came from.

What you should see

The timing features — flow inter-arrival time, idle time, flow duration — will probably lead, because waiting costs the attacker nothing.

If a Fixed feature appears in that list, your constraint check has a bug. Find it before going further.

8. Part D: Does SHAP point where the attacker goes?

SHAP tells you which features the model leans on. Part C tells you which features an attacker can reach. Where the two lists disagree is the interesting part.

Step D1: Put the two lists next to each other

What to write

SHAP values for 500 test flows with `TreeExplainer`, keeping class 1, ranked by mean absolute value.

Print the SHAP top ten beside the attacker's top ten from Step C3, count how many appear in both, and mark which group each SHAP feature belongs to.

Hint

`sv[:, :, 1]` picks the attack class out of the array the explainer returns.

TreeSHAP is slow on a large forest with many features, so keep to a subsample. 500 flows is plenty.

What you should see

A smaller overlap than you saw on the phishing data.

Several of the highest-ranked SHAP features are likely to be Fixed ones, which means the model is leaning on evidence the attacker cannot forge. That is good news.

Check yourself

Are your model's most important features ones the attacker can change, or ones they cannot? What does your answer say about how easily this model could be evaded?

If a feature is used often by the attack but ranked low by SHAP, what could explain that?

Step D2: Is the explanation stable under noise?

What to write

SHAP rankings for 200 flows on clean data, then for the same 200 flows with 5% noise added.

The Spearman rank correlation between the two rankings (from `scipy.stats.spearmanr`), plus both top-five lists.

What you should see

A correlation above 0.90 means the ranking is stable. Remember from Section 3.5 that stable is not the same as faithful — Part F tests the other half.

9. Part E: Two defences

Step E1: An ensemble

If one model is fooled by a change to a feature, a model with a different decision boundary may not be.

What to write

A gradient boosting classifier, 200 trees, depth 4, trained on the same training set.

An ensemble that averages the attack probability from the forest, the gradient boosting model and the logistic regression. Wrap it in a small class with `predict_proba` and `predict` so your `report()` still works.

Compare the forest with the ensemble on four things: clean data, 5% noise, 10% missing values, and the constrained attack from Step C2.

Hint

Gradient boosting takes several minutes on CICIDS. If it is too slow, cut the training set to 100,000 rows and say in your report that you did.

Check yourself

Did the ensemble help more against noise or against the attack? Why might those two differ?

An intrusion detector has to answer in milliseconds at line rate. Is running three models affordable there?

Step E2: Drop one feature, then drop a whole group

The usual test — remove one feature and see what breaks — is misleading here, for the reason in Section 3.6. Do it both ways.

What to write

Remove one feature, retrain a 100-tree forest, measure the test macro-F1. Repeat for every feature and print the five whose removal costs most.

Then find the groups from the data: `X_train.corr().abs()`, keeping pairs above 0.95. Remove each whole group in turn and measure again.

Hint

Use a subsample so the loop finishes in about five minutes, and use the same subsample every time so the comparison is fair.

What you should see

Single removals will look almost free, because a twin takes over.

Group removals will cost much more. The difference between the two is the redundancy in your feature set.

Check yourself

The single-feature test said a feature was expendable; the group test said its information was essential. Which answer do you give an engineer who is deciding what to stop collecting?

10. Part F: Are the explanations honest?

A SHAP chart looks convincing whether or not it is right, so it has to be tested. Three checks.

Step F1: The deletion test, alone and in groups

What to write

Take the 50 correctly detected attack flows from Part C.

For $k = 1, 3$ and 5 : replace the k highest-ranked SHAP features with training medians and measure how far the attack probability falls. Do the same for k random features, and print both averages.

Then replace the top-ranked feature together with everything correlated with it above 0.95 , and measure the fall again.

Hint

`np.argsort(-shap_row)[:k]` gives the positions of the k highest-ranked features for one flow.

Top- k against random- k measures fidelity. Single against group measures redundancy.

What you should see

Top- k should beat random- k clearly. If it only just beats it, the ranking is not describing the model well.

The group removal should push the probability down much further than the single removal.

Check yourself

Removing the top feature on its own barely moved the score, which makes it look as though SHAP was wrong about it. The group test shows the information was still there in its twin. So was SHAP wrong, or was the test?

Step F2: Do the numbers add up?

SHAP is built so that the base value plus all the SHAP values equals the model's output. It holds by construction for TreeSHAP; checking it is how you learn what the numbers actually are.

What to write

For any five test flows: the base value, the sum of the attack-class SHAP values, their total, and the model's predicted probability. Confirm they agree to within 0.001 .

Hint

TreeSHAP returns **log-odds**, not probabilities. Put the total through $1 / (1 + \exp(-x))$ before you compare it with `predict_proba`.

The two pieces you need are `sv.base_values[i, 1]` and `sv.values[i, :, 1].sum()`.

Check yourself

If the totals did not agree, what would that tell you about the explainer?

Step F3: How well does LIME fit?

LIME fits a straight line near one flow and gives you its slopes. If the line fits the model badly, the slopes mean very little — and LIME reports the quality of that fit, so you can check it.

What to write

A LIME tabular explainer on the training data with a fixed seed, run on 20 randomly chosen test flows.

Each fit score, plus the mean and the minimum, and the predicted probability of the worst-fitting flow.

Hint

`exp.score` is the R^2 of the local fit. Near 1.0 is good; below 0.70 the explanation should not be trusted much.

Flow features run from bytes in the millions down to ratios near zero, so leave `discretize_continuous=True` and let LIME work in ranges.

Check yourself

Were the worst-fitting flows the ones with a probability near 0.5 ?

Of these three checks, which could you run automatically on every alert an analyst sees, and why?

11. What to hand in

Your code: one notebook that runs from top to bottom, plus a short README (libraries, dataset, how to run). Keep `random_state=42`. **A short report (2–3 pages)** with these headings:

Heading	What to write
1. Problem	One paragraph: attack detection on flows, why a high score can be misleading, and what robustness means here.
2. Setup	Dataset and splits, the attack rate, the models, and your three feature groups from Step C1 (a short table is fine).
3. Results	The noise and missing-value tables (B1, B2); the constrained and unconstrained attack side by side (C2); the two top-ten lists and their overlap (D1); the ensemble and group-removal results (E1, E2); the three fidelity checks (F1–F3).
4. Discussion	How much of your score is real? How much of your robustness comes from the network rather than the model? Does your model lean on evidence an attacker can fake? Is your explanation stable, faithful, both or neither? Answer the “Check yourself” questions here.
5. Who did what	One or two lines.

How your work is graded

Every lab is graded 3 / 4 / 5 / F. You do not need to be perfect to pass: the work must run, make sense, and be clearly reported.

Criterion	Weight	What good work looks like here
Implementation and code quality	30%	The notebook runs start-to-finish and reproduces your numbers; the noise, missing-value, attack, ensemble and fidelity steps all work; README; seed is fixed.
Experimental design and methodology	25%	Noise and medians measured on the training set only; the three feature groups justified and enforced, with no Fixed feature ever changed; the attack run both constrained and unconstrained; the same subsample used across the removal loops.
Results and analysis	20%	Complete tables; the trivial baseline reported next to the model; an honest verdict on which model is most robust; the group-removal and group-deletion results compared with the single-feature ones.
Report quality	25%	Clear 2–3 pages; captioned figures and tables used in the text; who-did-what line included + screenshot of your code.

Submitting your work

- **Where.** Upload to CANVAS by the deadline: your code (a zip or a repository link) and the report as a PDF. Late submissions are not accepted in general.
- **Group work.** Add a short line saying who did what. If one person does everything, that shows up in individual grades.
- **Second chance.** If a lab does not reach grade 3 you may resubmit it once, before the final reporting date.
- **Honesty.** You may use libraries, tutorials and public code, just cite them. If you use an AI assistant, say so in the report.